

Graphify Report — Cargo Delivery Service

Generated: 2026-06-14
Repository: D: **Analysis Scope:** Full codebase (architecture docs, frontend source, mock data, status mapping engine)

Executive Summary

The Cargo Delivery Service is a centralized multi-carrier logistics platform. It unifies 4 major carriers (DHL, FedEx, UPS, Chronopost) under a single adapter interface with a sophisticated status mapping engine that translates 64+ carrier-specific status codes into 12 unified internal codes across 3 matching strategies (exact, regex, fuzzy). The frontend is a React + Vite + Tailwind TypeScript SPA with full CRUD on all entities, JWT-mock authentication, and multilingual support (EN/FR/TR).

The graph contains **40 nodes** and **48 edges** organized into **4 communities**.

Graph Statistics

Metric	Value
Total Nodes	40
Total Edges	48
Communities	4
EXTRACTED edges	43 (89.6%)
INFERRED edges	5 (10.4%)
Avg. confidence score	0.949

Community Map

Community 0 — Architecture & Carrier Adapters

Color: Blue (#3b82f6)
Nodes: 7

The architectural backbone. A unified `CarrierAdapter` interface is implemented by 4 carrier-specific adapters (DHL, FedEx, UPS, Chronopost), each normalizing carrier-specific concepts into a shared `UnifiedStatusModel`. The `CARGO-SERVICE-ARCHITECTURE.md` document defines the full contract.

- **God Node:** `cargo_service_arch` (degree: 5) — references endpoint docs, defines carrier adapter, connects to all adapter nodes
- **Key Pattern:** Pluggable adapter pattern — adding a new carrier requires only implementing the interface

Community 1 — Status Mapping Engine

Color: Green (#10b981)

Nodes: 6

The most technically sophisticated component. `StatusMapperEngine` maintains per-carrier maps with 3-tier matching (exact string → regex patterns → fuzzy substring fallback). It detects terminal vs. blocking states, builds milestone timelines, and verifies lifecycle transitions.

Carrier	Codes Mapped	Strategy Count
DHL	15	Exact (8) + Regex (5) + Fuzzy (2)
FedEx	16	Exact (9) + Regex (5) + Fuzzy (2)
UPS	18	Exact (10) + Regex (6) + Fuzzy (2)
Chronopost	15	Exact (7) + Regex (5) + Fuzzy (3)

- **God Node:** `status_mapper` (degree: 6) — central hub connecting to all carrier maps + milestone builder + unified status model
- **God Node:** `unified_status` (degree: 5) — used by all 4 adapters and the mapper

Community 2 — Frontend Application

Color: Yellow (#f59e0b)

Nodes: 21

The largest community. A React 18 + TypeScript SPA built with Vite and Tailwind CSS, organized around a service-layer architecture.

Layer Architecture:

Pages (dashboard, shipments, carriers, etc.)

- Shared Components (Sidebar, DataTable, Modal, etc.)
- React Hooks (`useApi`, `useMutation`)
- Cargo Service Layer (20+ API functions)
 - Mock Data (6 entity stores)
 - Status Mapper Engine

Key Metrics: - 9 page components, all with full CRUD modals - 6+ shared/reusable components - 4 form components (ShipmentForm, CarrierForm, PickupForm, ServiceForm) - AuthGuard route protection wrapping all pages - 750+ i18n keys across 3 locales

God Nodes: - react_frontend (degree: 6) — root dependency hub -
cargo_service_layer (degree: 9) — most highly connected node in the graph, used by every page - auth_system (degree: 3) — crosscuts both frontend and REST endpoint design

Community 3 — API Design & Documentation

Color: Purple (#8b5cf6)

Nodes: 7

Documents the RESTful API contract covering 18 endpoints organized around 6 functional areas: - Shipment Lifecycle (7 states: draft → cancelled/delivered) - Carrier CRUD (add/edit/delete carriers) - Rate Engine (quote calculation) - Pickup Flow (schedule/cancel pickups) - Webhook Simulation (trigger/receive webhooks) - Address Validation

God Node: rest_endpoints (degree: 6) — documents all functional areas

Endpoint Reference (18 endpoints)

#	Method	Path	Purpose	Fr
1	POST	/api/v1/cargo/auth/login	Authenticate (mock JWT)	Login
2	POST	/api/v1/cargo/rates	Get shipping rates	Rates
3	POST	/api/v1/cargo/shipments	Create shipment	Shipr
4	GET	/api/v1/cargo/shipments/{id}	Get shipment details	Shipr
5	GET	/api/v1/cargo/shipments/{id}/tracking	Full tracking with events	Shipr → Tr
6	POST	/api/v1/cargo/shipments/{id}/cancel	Cancel shipment	Shipr
7	POST	/api/v1/cargo/shipments/{id}/label	Generate label	Shipr
8	POST	/api/v1/cargo/pickups	Schedule a pickup	Picku
9	POST	/api/v1/cargo/webhooks/{carrierCode}	Receive carrier push updates	Webhook
10	POST	/api/v1/cargo/addresses/validate	Validate & normalize address	Shipr form
11	POST	/api/v1/cargo/carriers	Add new carrier	Carri

#	Method	Path	Purpose	Fr
12	GET	/api/v1/cargo/carriers	List all carriers	Carri
13	GET	/api/v1/cargo/carriers/{code}	Get carrier detail	Carri
14	PUT	/api/v1/cargo/carriers/{code}	Update carrier settings	Carri
15	PATCH	/api/v1/cargo/carriers/{code}/toggle	Enable/disable carrier	Carri
16	POST	/api/v1/cargo/carriers/{code}/test	Test carrier API connection	Carri
17	POST	/api/v1/cargo/carriers/{code}/services	Add a carrier service	Carri
18	PUT	/api/v1/cargo/carriers/{code}/credentials	Update carrier API keys	Carri

Frontend ↔ Endpoint Mapping

Login (Page)	→ auth/login
Rates (Page)	→ cargo/rates
Shipments (Page)	→ cargo/shipments (CRUD list + create)
ShipmentDetail (Page)	→ cargo/shipments/{id} (get, cancel, label, tracking)
Carriers (Page)	→ cargo/carriers (list + add)
CarrierDetail (Page)	→ cargo/carriers/{code} (detail, edit, toggle, test, services, credentials)
Pickups (Page)	→ cargo/pickups (schedule, cancel)
WebhookLogs (Page)	→ cargo/webhooks/{carrierCode} (view logs)
Dashboard (Page)	→ aggregates data from shipments + carriers (stats)

Mock Service Functions (20+ in cargoService.ts)

Function	Endpoint	Type
login()	POST auth/login	Query
getRates(req)	POST cargo/rates	Query
getShipments()	GET cargo/shipments	List
getShipmentById(id)	GET cargo/shipments/{id}	Detail
createShipment(req)	POST cargo/shipments	Mutati
deleteShipment(id)	DELETE cargo/shipments/{id}	Mutati
updateShipment(id, data)	PUT cargo/shipments/{id}	Mutati
cancelShipment(id)	POST cargo/shipments/{id}/cancel	Mutati
getTrackingData(id)	GET cargo/shipments/{id}/tracking	Detail
getCarriers()	GET cargo/carriers	List

Function	Endpoint	Type
getCarrierByCode(code)	GET cargo/carriers/{code}	Detail
addCarrier(data)	POST cargo/carriers	Mutati
updateCarrier(code, data)	PUT cargo/carriers/{code}	Mutati
deleteCarrier(code)	DELETE cargo/carriers/{code}	Mutati
toggleCarrier(code, active)	PATCH cargo/carriers/{code}/toggle	Mutati
testCarrierConnection(code)	POST cargo/carriers/{code}/test	Query
addCarrierService(code, data)	POST cargo/carriers/{code}/services	Mutati
deleteCarrierService(code, svcCode)	DELETE cargo/carriers/{code}/services/{svcCode}	Mutati
updateCarrierCredentials(code, data)	PUT cargo/carriers/{code}/credentials	Mutati
getPickups()	GET cargo/pickups	List
schedulePickup(data)	POST cargo/pickups	Mutati
cancelPickup(id)	POST cargo/pickups/{id}/cancel	Mutati
getWebhookLogs()	GET cargo/webhooks/logs	List
validateAddress(req)	POST cargo/addresses/validate	Query
getDashboardStats()	GET cargo/stats/dashboard	Detail

God Nodes (High-Connectivity Hubs)

Node	Community	Degree	File Type
cargo_service_layer	Frontend	9	code
cargo_service_arch	Architecture	5	document
unified_status	Architecture	5	code
status_mapper	Status Mapping	6	code
react_frontend	Frontend	6	code
rest_endpoints	API Design	6	document
shared_components	Frontend	5	code

The **cargo_service_layer** is the single most critical node — every page depends on it.
The **status_mapper** is the most technically complex.

Edge Analysis

Strongest Relations (EXTRACTED, confidence = 1.0)

43 edges are directly extracted from the codebase (89.6%). These include: - depends_on — frontend layer hierarchy - contains — mapper → maps, arch → adapter - implements — adapter interface → carrier implementations - uses — pages → service layer, forms → pages - part_of — components → frontend framework - maps_to — status mapper → unified status model

Inferred Relations (confidence < 1.0)

5 edges are semantically inferred (10.4%): - complements — status_mapper ↔ carrier_adapter (0.85) - crosscuts — i18n_system → react_frontend (0.95) - links_to — dashboard → shipments (0.95), shipments → tracking (1.0) - competes_with — DHL ↔ FedEx (0.85), UPS ↔ Chronopost (0.75) - required_by — auth_system → endpoints (0.85)

Orphan & Overlap Analysis

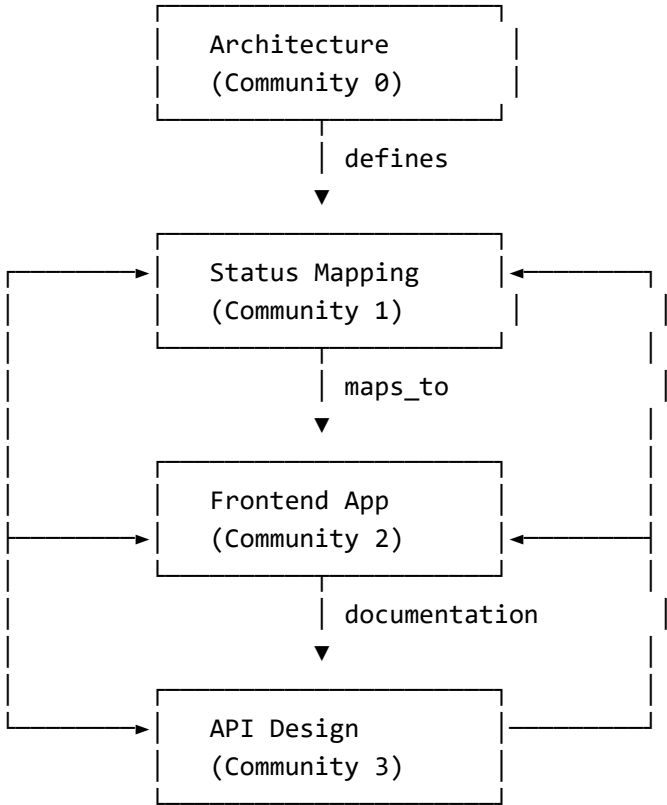
Orphans: 0 — all nodes have at least 1 edge.

Overlaps: The auth_system node bridges Community 2 (Frontend) and Community 3 (API Design), as authentication crosscuts both frontend routing and backend endpoint security.

Key Architectural Insights

1. **Pluggable Carrier Architecture** — Adding a 5th carrier requires: (a) implement adapter interface, (b) add status map entries, (c) add mock data rows. No changes to existing code.
2. **Status Mapping is the Core Differentiator** — 3-tier matching with regex means the system handles real-world carrier status variations (typos, locale differences, formatting). Turkish carrier status names are supported via regex patterns.
3. **Service Layer as God Node** — The single cargoService.ts module is the bottleneck. Consider splitting into domain-specific services (shipmentService.ts, carrierService.ts, etc.) for better maintainability.
4. **Frontend Outweighs Backend** — 21 of 40 nodes (52.5%) are frontend-related. The “backend” is currently mock data, but the architecture is ready for real API integration by swapping cargoService.ts implementations.
5. **Documentation is Centralized** — 3 service documentation files (EN, FR, color-coded) cover the same API spec, creating potential sync drift. Consider auto-generating from a single source.

Community Cross-Reference



File Details

Core Documents

File	Purpose
CARGO-SERVICE-ARCHITECTURE.md	Full backend architecture plan
service_cargo.md	API endpoint docs (French)
cargo_service.md	API endpoint docs (English)
cargo_service_api.md	Color-coded API reference

Frontend Source

File/Dir	Purpose
frontend/src/types/index.ts	14 TypeScript interfaces
frontend/src/data/mockData.ts	6 entity mock stores
frontend/src/services/statusMapper.ts	Status mapping engine (4 carriers)
frontend/src/services/cargoService.ts	20+ mock API functions
frontend/src/hooks/useCargoService.ts	useApi, useApiById, useMutation, useLoading

File/Dir	Purpose
frontend/src/context/AuthContext.tsx	JWT mock auth state
frontend/src/i18n/	EN/FR/TR translation system
frontend/src/pages/	9 page components
frontend/src/components/	10+ shared components
frontend/src/components/forms/	4 form components

Report generated by Graphify knowledge graph pipeline.